UNIVERSIDAD DE MANIZALES
Facultad de Ciencias e Ingenierías

Documento de Decisiones de Diseño

Sistema de Transporte
Actividad Colaborativa N.° 3 —
Proyecto Final: Desarrollo de un Sistema Orientado a Objetos

Integrantes
Omar Alejandro Gómez
Juan Carlos Girón Sanabria
Cristhian Mauricio Hernández Gaviria

Docente
NATALIA MARCELA CASTELLANOS GOMEZ

Repositorio: github.com/OmarAlejandroGomez/Actividad03_ProyectoFinal

Manizales, noviembre de 2026

1. Introducción

Quien usa un sistema de transporte masivo espera que la tarjeta funcione, que el bus llegue y que el cobro sea correcto. Detrás de esa experiencia hay reglas que el sistema debe verificar en cada viaje, esto es, que la ruta esté operando a esa hora, que las estaciones de origen y destino pertenezcan a la ruta del vehículo, que la tarjeta no esté bloqueada y que tenga saldo suficiente.

Además, no todos los vehículos se comportan igual toda vez que un bus troncal cobra la tarifa completa y avanza por un corredor de varias estaciones, un alimentador recoge pasajeros en los barrios sin cobrarles, y un cable aéreo va directo hasta su única parada.

El equipo eligió el contexto de sistema de transporte y lo concretó en un problema específico: administrar la operación de una red de transporte masivo, validando y registrando cada viaje que hace un pasajero con su tarjeta.

El sistema registra estaciones, rutas con su horario de operación, la flota de vehículos y los pasajeros con sus tarjetas; decide si un viaje se puede hacer y, cuando se puede, cobra la tarifa que corresponde al tipo de vehículo, calcula el tiempo estimado del recorrido y deja constancia del viaje. Con esa información genera el recaudo total, el número de ingresos por estación y las próximas salidas de cada ruta.

Este documento explica cómo se diseñó esa solución en Python con programación orientada a objetos, qué decisiones se tomaron, cuáles se cambiaron en el camino y por qué. Todas las referencias a clases, atributos y métodos corresponden al código entregado en el repositorio.

2. Descripción general de la solución

La solución se organizó en tres capas, cada una en su propia carpeta.

La carpeta models contiene: estaciones, rutas, vehículos, tarjetas, pasajeros, viajes y el propio sistema de transporte.

La carpeta controllers contiene dos elementos que administran colecciones de pasajeros y de tarjetas y evitan duplicados.

Por último, el archivo main.py es el programa principal, este ofrece un menú de 19 opciones para operar el sistema y una opción de demostración que crea los objetos de prueba, ejecuta los casos normales y los excepcionales y presenta los informes.

La clase central es SistemaTransporte, que reúne las estaciones, las rutas, la flota y los viajes, y concentra la operación más importante del programa, realizar_viaje().

Ese método valida el horario de la ruta, cuenta los tramos entre origen y destino, pide al vehículo su tarifa y su tiempo estimado, cobra a la tarjeta, registra el ingreso en la estación de origen y crea el objeto Viaje.

El orden de esos pasos es deliberado: las validaciones se hacen antes del cobro, de modo que si el viaje no es posible, la tarjeta nunca se debita.

El diagrama de clases del sistema se encuentra en el repositorio como docs/diagrama_clases.drawio.

3.1. Clases principales y responsabilidades

Para definir las clases, el equipo partió de los sustantivos del problema (estación, ruta, vehículo, pasajero, tarjeta, viaje) y le asignó a cada uno la información que le pertenece y las acciones que solo él puede realizar.

Así, el saldo existe en la tarjeta y solo la tarjeta lo modifica; el conteo de ingresos se encuentra en la estación; y la decisión de cuánto cobrar y cuánto tarda un recorrido le corresponde a cada tipo de vehículo.

La Tabla 1 resume la responsabilidad de cada clase.

Clase	Responsabilidad
SistemaTransporte	Reúne estaciones, rutas, flota y viajes; registra cada elemento evitando duplicados; realiza los viajes y genera el recaudo y el reporte de estaciones.
Estacion	Guarda código, nombre, zona y accesibilidad, y cuenta los ingresos de pasajeros con registrar_ingreso().
Ruta	Define el recorrido como una secuencia ordenada de al menos dos estaciones sin repetir; cuenta los tramos entre dos estaciones y valida el horario.
ProgramacionHoraria	Guarda la hora de inicio, la hora de fin y la frecuencia de una ruta; dice si la ruta opera a una hora dada y calcula las próximas salidas.

Clase	Responsabilidad
Vehiculo (abstracta)	Define lo común a todo vehículo: código, capacidad, ruta asignada y el contrato de calcular_tarifa() y tiempo_estimado().
BusTroncal	Vehículo de corredor troncal, con número de cuerpos y lado de las puertas.
BusAlimentador	Vehículo que atiende un barrio con un número de paradas fijas.
CableBus	Cabina de cable aéreo que va directo a una única parada en un tiempo fijo.
Pasajero	Guarda documento, nombre y perfil, y es titular de máximo una tarjeta.
Tarjeta	Maneja el saldo y el bloqueo; recarga, valida el pago y cobra la tarifa.
Viaje	Registra un viaje realizado: tarjeta, vehículo, origen, destino, hora, valor cobrado y minutos estimados.
PasajeroController y TarjetaController	Administran las colecciones de pasajeros y tarjetas, buscan por documento o número e impiden registros duplicados.
TransporteError y sus hijas	Representan las situaciones anormales del dominio (ver sección 3.8).

3.2. La clase abstracta Vehículo

Vehiculo se definió como clase abstracta heredando de ABC y marcando con @abstractmethod los métodos calcular_tarifa(tarifa_base) y tiempo_estimado(tramos).

La razón es de dominio: en el sistema no existe un "vehículo" genérico que un pasajero pueda abordar. Siempre se aborda un bus troncal, un alimentador o un cable, y cada uno tiene su propia regla de cobro y de tiempo. Si se pudiera crear un Vehiculo directamente, el sistema no sabría cuánto cobrarle al pasajero ni cuánto tardaría el recorrido.

El código planteado hace cumplir esta decisión toda vez que al intentar Vehiculo("X", 10) se produce un TypeError que indica que la clase no implementa los métodos abstractos. La clase abstracta no está vacía, sin embargo esta concentra el código que sí es igual para todos los vehículos, como la validación del código y de la capacidad, la asignación de la ruta y la parte común de descripcion().

3.3. Herencia y especialización

La relación de herencia implementada es BusTroncal, BusAlimentador y CableBus a Vehiculo.

Las tres cumplen la prueba de "es un". En este sentido un bus troncal es un vehículo, un alimentador es un vehículo y un cable bus es un vehículo del sistema. Los tres

transportan pasajeros, tienen código y capacidad, se asignan a una ruta y se usan en un viaje. Por eso SistemaTransporte los guarda en la misma flota y realizar_viaje() los trata a todos igual.

Las tres hijas heredan los atributos _codigo, _capacidad y _ruta, y los métodos get_codigo(), set_codigo(), get_capacidad(), set_capacidad(), get_ruta(), asignar_ruta() y la versión base de descripcion().

Cada constructor hijo llama a super().__init__(codigo, capacidad) para reutilizar esa validación en lugar de repetirla.

Lo que distingue a cada hija está en la Tabla 2.

Clase hija	Atributos propios	Métodos propios
BusTroncal	_cuerpos (1 sencillo, 2 articulado, 3 biarticulado) y _puertas_izquierdas	lado_de_abordaje(), que indica si se aborda por la izquierda o por la derecha
BusAlimentador	_barrio y _paradas_fijas	cambiar_barrio(nuevo_barrio), para reasignar el barrio que atiende
CableBus	_parada (su única parada) y _minutos_trayecto	llega_a(nombre_estacion), que dice si una estación es su parada

El tercer tipo, CableBus, se agregó durante el desarrollo teniendo en cuenta el requerimiento del trabajo. Para incluirlo, se tuvo en cuenta que el cable aéreo es un medio real de los sistemas de transporte colombianos, tiene información propia (una sola parada y un tiempo de trayecto fijo) y un comportamiento distinto de los dos buses.

3.4. Métodos sobrescritos

Las tres clases hijas sobrescriben los mismos tres métodos de Vehiculo. El nombre y los parámetros se conservan, pero el resultado cambia según el tipo de objeto, como muestra la Tabla 3.

Método	BusTroncal	BusAlimentador	CableBus
calcular_tarifa(tarifa_base)	Cobra la tarifa base del sistema completa.	Devuelve 0: no cobra, solo registra el ingreso.	Devuelve 0: cobra lo mismo que el alimentador.
tiempo_estimado(tramos)	3 minutos por cada tramo.	5 minutos por tramo más 1 minuto por	Devuelve sus _minutos_trayecto,

Método	BusTroncal	BusAlimentador	CableBus
		cada una de sus _paradas_fijas.	sin importar los tramos, porque va directo.
descripcion()	Agrega el tipo, los _cuerpos y el lado de abordaje.	Agrega el tipo, el _barrio y las _paradas_fijas.	Agrega el tipo, la _parada única y los _minutos_trayecto.

En la demostración, con una tarifa base de $3.000, un viaje de tres tramos en el bus troncal cuesta $3.000 y tarda 9 minutos; uno de un tramo en el alimentador con 6 paradas fijas cuesta $0 y tarda 11 minutos; y el cable bus cuesta $0 y tarda los 13 minutos de su trayecto. descripcion() sobrescrita llama primero a super().descripcion() para reutilizar la parte común y luego agrega los datos propios de cada clase.

3.5. Polimorfismo en el programa principal

El polimorfismo se evidencia en dos lugares. El primero es la función mostrar_flota() de main.py, en tanto se obtiene la flota con sistema.get_flota(), una sola colección que contiene buses troncales, alimentadores y cables, y la recorre con un único ciclo for que invoca vehiculo.descripcion() sobre cada objeto.

No hay ningún if que pregunte de qué tipo es el vehículo; cada objeto responde con su propia versión del método.

El segundo está dentro del sistema SistemaTransporte.realizar_viaje() que llama a vehiculo.calcular_tarifa() y a vehiculo.tiempo_estimado() sin saber si el vehículo es troncal, alimentador o cable.

Gracias a eso, agregar CableBus no requirió modificar SistemaTransporte fue suficiente con crear la clase hija. En main.py sí existe un isinstance en elegir_alimentador(), pero no decide qué método ejecutar, este solo filtra la lista para la opción “Cambiar barrio”, que únicamente tiene sentido para un alimentador porque cambiar_barrio() es un método propio de esa clase.

3.6. Composición: la ruta y su programación horaria

La composición se aplicó entre Ruta y ProgramacionHoraria. El constructor de Ruta no recibe un objeto de programación ya creado. Recibe la hora de inicio, la hora de

fin y la frecuencia, y crea su propia programación con self._programacion = ProgramacionHoraria(hora_inicio, hora_fin, frecuencia).

Esa programación nace con la ruta, no se comparte con ninguna otra y no tiene sentido sin ella en tanto un horario de "05:00 a 22:00 cada 10 minutos" no significa nada si no se sabe de qué ruta es.

Si la ruta deja de existir, su programación desaparece con ella. Ese ciclo de vida compartido es lo que define la composición.

Lo anterior, no se resolvió con herencia porque una ruta no es una programación horaria puesto que una ruta tiene una programación horaria. Si Ruta heredara de ProgramacionHoraria, la ruta quedaría expuesta como si fuera un horario, con métodos como set_horas() o proximas_salidas() mezclados con sus propias responsabilidades, y el diseño no se ajustaría al dominio.

Separar la programación en su propia clase, en cambio, permite que valide por su cuenta que la hora de fin sea posterior a la de inicio y que la frecuencia sea un entero positivo, mientras Ruta solo le delega la pregunta con validar_horario(hora).

Esta relación contrasta intencionalmente con la que hay entre Ruta y Estacion, que es una agregación.

Las estaciones se crean antes que las rutas, se registran en el sistema de forma independiente y una misma estación puede pertenecer a varias rutas (en la demostración, "Portal Norte" está en la ruta troncal y en la alimentadora). Por eso Ruta recibe las estaciones ya existentes y solo guarda una copia de la lista.

3.7. Otras relaciones entre objetos y encapsulamiento

Además de la herencia y la composición, el sistema tiene varias asociaciones. Un Pasajero es titular de cero o una Tarjeta, y vincular_tarjeta() impide asignarle una segunda.

Un Vehiculo recorre una Ruta que se le asigna al registrarlo en el sistema.

Un Viaje relaciona una tarjeta, un vehículo y dos estaciones (origen y destino).

SistemaTransporte agrega estaciones, rutas, vehículos y viajes, mientras que los controladores agregan pasajeros y tarjetas.

En cuanto al encapsulamiento, los atributos de las entidades se declararon con el prefijo _, que en Python indica que son de uso interno, y se accede a ellos mediante métodos get_ y set_.

Los métodos set_ validan antes de asignar por lo que no se aceptan códigos ni nombres vacíos, capacidades menores o iguales a cero, ni frecuencias que no sean enteras positivas.

Algunos atributos no tienen set_ a propósito como el saldo de la tarjeta solo cambia con recargar() y cobrar(), y los ingresos de la estación solo aumentan con registrar_ingreso(), para que nadie pueda asignar un valor arbitrario.

Las colecciones de SistemaTransporte y de los controladores se entregan siempre como copias (get_estaciones(), get_flota() y los demás devuelven .copy()), de modo que desde main.py no se pueda alterar la lista interna sin pasar por los métodos de registro.

3.8. Excepciones personalizadas

El equipo identificó cuatro situaciones anormales propias del dominio y creó una excepción para cada una. Todas heredan de una clase base común, TransporteError, que a su vez hereda de Exception. Esto permite capturarlas por separado o como grupo. Cada excepción guarda los datos del problema como atributos (por ejemplo, el saldo actual y la tarifa requerida) y construye un mensaje claro para el usuario.

Excepción	Situación que controla	Método que la lanza
TarjetaBloqueadaError	Se intenta viajar con una tarjeta bloqueada, por ejemplo reportada como perdida. Aplica incluso si el vehículo no cobra.	Tarjeta.cobrar()
SaldoInsuficienteError	El saldo de la tarjeta no alcanza para pagar la tarifa.	Tarjeta.cobrar()
EstacionNoPerteneceARutaError	El origen o el destino no forman parte de la ruta del vehículo.	Ruta.contar_tramos()
FueraDeHorarioError	Se intenta viajar a una hora en que la ruta no opera.	SistemaTransporte.realizar_viaje()

Cada excepción se lanza desde el método que tiene la información para detectar el problema de esta manera la tarjeta conoce su saldo y su estado, la ruta conoce sus estaciones y el sistema consulta el horario antes de cualquier otra validación.

En main.py, las funciones realizar_viaje() e intentar_viaje() capturan cada excepción en un bloque except separado, muestran un mensaje que explica por qué se rechazó el viaje y devuelven el control al menú o a la demostración.

La demostración comprueba esto en su paso 5 el cual provoca las cuatro situaciones una tras otra, muestra los cuatro mensajes y continúa hasta el final, donde imprime "La demostración terminó sin interrupciones".

3.9. Reglas de negocio supuestas

Algunas reglas no estaban dadas por la actividad y el equipo las definió para que el sistema funcionara de forma coherente. Se dejaron documentadas como comentarios en el código: el troncal cobra la tarifa base y tarda 3 minutos por tramo; el alimentador no cobra y tarda 5 minutos por tramo más 1 por parada fija; el cable bus cobra lo mismo que el alimentador y tarda un tiempo fijo. Además, todas las tarjetas pagan la tarifa completa sin importar el perfil del pasajero, y una tarjeta bloqueada no puede usarse aunque el viaje sea gratuito, porque el bloqueo existe para evitar el uso de tarjetas perdidas.

3.10. Decisiones modificadas durante el desarrollo

El diseño cambió varias veces mientras se construía. La estructura también evolucionó con la creación de la base del sistema (SistemaTransporte, Vehiculo, Viaje, Ruta, Estacion); posteriormente la creación de las clases de bus, Pasajero y Tarjeta; seguidas de la creación de la administración de pasajeros y tarjetas en la carpeta controllers y de las excepciones.

En un avance posterior, la información de horario se separó en la clase ProgramacionHoraria, lo que dio lugar a la composición descrita en la sección 3.6.

La revisión del proyecto contra los requerimientos llevó a decisiones importantes. Se detectó que solo había dos clases hijas de Vehiculo, así que se agregó CableBus como tercer tipo y se integró al menú y a la demostración.

-

3.11. Uso de herramientas de inteligencia artificial

El equipo utilizó **Claude** (Anthropic) como asistente durante el desarrollo. Se usó para algunas tareas como revisar el código frente a los requerimientos de la actividad, ejecutándolo para encontrar los errores listados en la sección 3.10, así como apoyar la redacción de este documento, que el equipo revisó y ajustó.

Desde la IA se realizaron propuestas de código y criterios, los cuales fueron objeto de analisis por parte de los integrantes, en aras de generar un programa claro y sencillo, de fácil compresión y seguimiento.

Se debe indicar que las correcciones sugeridas se aplicaron de forma puntual, línea por línea revisando su alcance y funcionamiento.

3.12. Verificación del funcionamiento

El equipo verificó que el código funcionara y correspondiera con los conceptos estudiados de cuatro maneras.

Primero, compiló todos los archivos para descartar errores de sintaxis.

Segundo, ejecutó la opción 18 del menú: se crean 5 estaciones, 3 rutas, 3 vehículos (uno de cada tipo), 4 pasajeros y 4 tarjetas; se realizan viajes válidos en los tres tipos de vehículo; se provocan las cuatro excepciones; se corrigen las causas (se desbloquea una tarjeta y se recarga otra) y se repiten los viajes; y se imprimen el recaudo total ($9.000), el reporte de estaciones y las próximas salidas.

Tercero, probó el menú a mano, registrando estaciones, rutas, un vehículo de cada tipo, pasajeros, tarjetas y viajes. Esa prueba fue la que reveló los errores de agregar_estacion() y del if/elif. Cuarto, comprobó que Vehiculo no se pudiera instanciar.

Para relacionar el código con los conceptos, el equipo revisó cada requerimiento de la actividad contra una ubicación concreta en el código, como se resume en el README. Además, cada vez que se integraban cambios de varios integrantes, se volvía a hacer git pull y a ejecutar la demostración.

4. Conclusiones

El proyecto mostró que los conceptos de programación orientada a objetos son decisiones de diseño con consecuencias prácticas. La clase abstracta y la sobrescritura permitieron que SistemaTransporte cobrara y calculara tiempos sin conocer el tipo de vehículo, y por eso agregar el cable bus no exigió tocar la lógica del sistema.

La composición entre ruta y programación horaria mantuvo cada clase pequeña y con una sola responsabilidad. Las excepciones personalizadas, lanzadas antes del cobro, garantizaron que un viaje inválido nunca debitara la tarjeta.

La principal dificultad fue la coordinación. Tres personas editando los mismos archivos produjeron conflictos al hacer push, nombres de atributos inconsistentes entre autores y errores que Python solo revela al ejecutar la línea donde están, como un atributo mal escrito o un método inexistente.

De ahí salieron los aprendizajes más útiles, que aún continúa, como el hacer pull antes de editar, acordar convenciones de nombres desde el inicio, ejecutar el programa completo después de cada integración y hacer correcciones puntuales respetando el trabajo de los demás y evitando reprocesos.

Por último, el uso de inteligencia artificial resultó útil para encontrar errores y documentar, pero cada propuesta tuvo que contrastarse con el diseño acordado por el equipo, y varias se modificaron o descartaron.

5. Referencias y herramientas

- Herramientas: Python 3, Visual Studio Code, Git y GitHub (control de versiones y trabajo colaborativo), draw.io (diagrama UML) y Claude, Gemini.